Guide Kibo-RPC 7 — Tim Kita

Oct 30, 2026 · @yarigai

Program misi kita sudah jadi dan, di uji tiruan memakai modul juri asli JAXA, versi v1.1 (dikalibrasi dari run Gazebo pertama di laptop) mengunjungi 8–9 dari 12 checkpoint level tersulit, semuanya Level 3, lalu Goal dalam ±250–270 detik (batas 300 detik). Yang paling mendesak: daftar tim sebelum Nov 4, 2026 jam 14:59 WIB.

Yang harus dilakukan

Pendaftaran dulu, baru setup. Program sudah ada, jadi setelah setup tinggal tes dan submit.

- [ ] Kumpulkan tim minimal 3 orang (mahasiswa/siswa aktif, satu orang hanya boleh di satu tim) dan tentukan nama tim
- [ ] Baca Guidebook (`01_Dokumen`), karena form menganggap kita setuju isinya
- [ ] Isi form pendaftaran Indonesia sebelum Nov 4, 2026 14:59 WIB, dan jangan isi ulang (perubahan lewat email ke POC BRIN)
- [ ] Install WSL2 + Ubuntu + Docker (bagian Setup di bawah)
- [ ] `build.sh` lalu jalankan simulator lokal dengan program kita
- [ ] Tunggu email ID/password dari POC (cek spam), lalu coba Web Simulator
- [ ] Submit program preliminary paling lambat Jan 20, 2027 23:59 JST (21:59 WIB); program harus berstatus FINISHED di simulator "Preliminary"

Jadwal besar: penyisihan 21 Jan – 17 Feb 2027, final run di ISS sekitar April 2027, acara final Juni 2027.

Aturan main dan skor

Robotnya Int-Ball2 (bukan Astrobee). Ia lepas dari docking station di (0.2, 0, 0), mengunjungi 6–12 checkpoint, lalu berakhir di checkpoint Goal (ID 12, dekat titik start). Batas waktunya 5 menit sejak Mission Start.

Hal	Isi
Jumlah checkpoint	easy 6 · normal 8 · hard 10 · veryhard 12 (selalu termasuk Goal)
Letak checkpoint	0.5 m di depan AR marker, arah acak ±20° yaw / ±15° pitch; roll target acak ±180°
Syarat lolos	3 syarat sekaligus, ditahan 1 detik; level = yang terendah dari ketiganya
Posisi	jarak ke titik checkpoint: Lv3 ⩽ 0.10 m · Lv2 ⩽ 0.20 m · Lv1 ⩽ 0.30 m
Centering kamera	sudut antara sumbu +X robot dan arah ke marker: Lv3 ⩽ 10° · Lv2 ⩽ 20° · Lv1 ⩽ 30°
Roll	beda putaran di sekitar garis pandang vs target: Lv3 ⩽ 10° · Lv2 ⩽ 20° · Lv1 ⩽ 30°
Skor akhir	(poin semua checkpoint + poin Goal) × pengali waktu (makin cepat lapor selesai, makin besar, maks ±2×)
Penyisihan	program dijalankan 10× dengan acakan berbeda, skor = rata-rata
Gangguan	modul "microgravity" mendorong robot (pola tidak diberitahu)

Tiga hal penting yang kita temukan di kode juri (`judge.py`), tidak tertulis jelas di Rulebook:

1. **Tiap checkpoint hanya dinilai sekali.** Laporan kedua untuk ID yang sama diabaikan, jadi jangan lapor sebelum posisi benar-benar pas.
2. **Juri menilai pakai `/sensor_fusion/navigation`** , topic yang sama yang kita baca. Artinya program bisa menghitung levelnya sendiri persis seperti juri sebelum melapor.
3. **Goal dianggap tercapai** kalau checkpoint Goal sudah dilaporkan lolos, atau kalau robot sudah ada di Goal saat `mission_finished` dipanggil.

Mengakses data jawaban di luar API (misalnya `cheat_sheet.json`) bisa ditandai pelanggaran oleh juri (`Illegal`). Program kita hanya memakai API resmi di bab 6 Programming Manual.

Setup laptop

Semua langkah resmi ada di Programming Manual bab 2; script di `05_Setup` menjalankan perintah yang sama. Syarat simulator lokal: Windows 11, CPU ≥ 4 core, RAM ≥ 8 GB (16 GB disarankan), disk kosong ≥ 20 GB.

1. **Install WSL2 + Ubuntu 24.04.** Klik 2× `05_Setup\0_install_wsl_ubuntu.bat` (minta izin Administrator). Restart kalau diminta, lalu buka "Ubuntu" dari Start Menu dan buat username + password. Cek di PowerShell: `wsl --list --verbose` harus menunjukkan VERSION 2.
2. **Jalankan setup di terminal Ubuntu:**

   ```
   bash "/mnt/e/KIBO Challenge/05_Setup/1_setup_ubuntu.sh"
   ```

   Script ini memasang Docker, git dan numpy, mengekstrak simulator ke `~/krpc/krpc7th-simulator`, lalu memasang program kita (template asli dibackup sebagai `user_template.original.py`).
3. **Tutup lalu buka lagi terminal Ubuntu**, kemudian cek: `docker run hello-world`.
4. **Build simulator (sekali saja, lama):**

   ```
   cd ~/krpc/krpc7th-simulator && bash ./build.sh
   ```

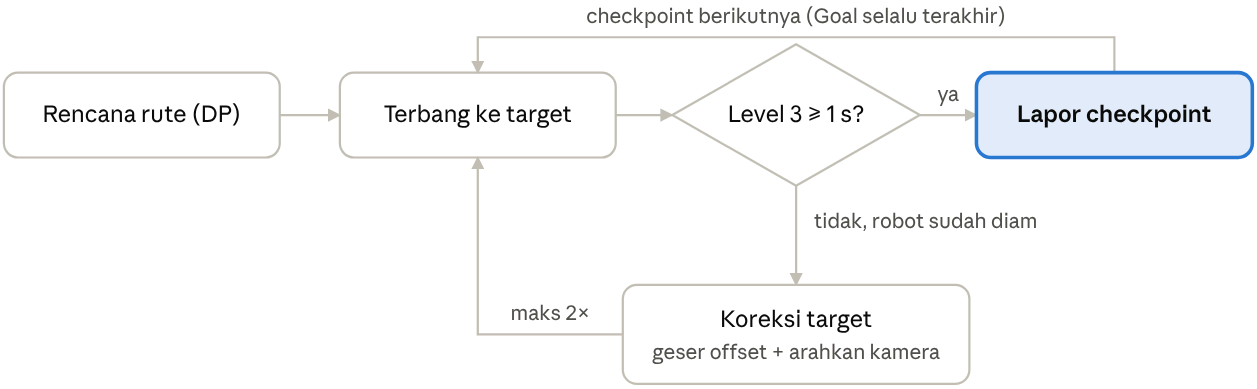
   Image-nya ±6 GB dan meng-compile OpenCV dari source, jadi siapkan 30–90 menit dan koneksi stabil.
5. **Jalankan simulasi:** `bash ./simulation.sh`, lalu buka http://localhost di browser untuk melihat robotnya. Hasil ada di `share/logs/latest/`, dan replay bisa dengan `bash ./replay.sh`.
6. **Editor:** VS Code + ekstensi "WSL", lalu Open Folder `~/krpc/krpc7th-simulator/users/`. Edit file di dalam Ubuntu, bukan di drive E:.

Kalau laptop tidak kuat, simulator lokal boleh dilewati. WSL + Ubuntu tetap wajib untuk membuat file submit, lalu tes di Web Simulator (maks 120 run/bulan per tim).

Cara kerja program kita

Program (`04_Solusi/user_template.py`) menghitung level sendiri dengan rumus yang sama seperti juri, lalu baru melapor saat Level 3 sudah stabil.

Tiap checkpoint dilaporkan sekali, setelah Level 3 bertahan 1 detik



alur per checkpoint · user_template.py

1. **Rute:** DP Held–Karp memilih kombinasi dan urutan yang mengunjungi checkpoint sebanyak mungkin tapi masih sempat ke Goal sebelum batas waktu (±0.02 detik hitung). Rute dihitung ulang sebelum tiap leg memakai kecepatan robot yang terukur, jadi checkpoint yang tadi dibuang bisa masuk lagi kalau robot ternyata lebih cepat. Goal selalu paling akhir.
2. **Terbang:** `/ctl/command` MOVE_TO_ABSOLUTE_TARGET ke posisi checkpoint. Orientasinya dihitung ulang supaya sumbu kamera (+X) tepat ke marker dan roll-nya sama dengan target.
3. **Nilai diri sendiri:** tiap 0.05 detik posisi dari `/sensor_fusion/navigation` dinilai: jarak, centering kamera, roll.
4. **Lapor sekali:** `checkpoint_passed` dikirim setelah Level 3 bertahan 1 detik (aturan Rulebook).
5. **Koreksi:** kalau robot sudah diam tapi belum Level 3, target posisi digeser sebesar selisihnya dan orientasi dihitung ulang secara absolut dari posisi robot sebenarnya, maksimal 2×. Offset posisi akibat gangguan juga dipelajari dan dikompensasi di checkpoint berikutnya.
6. **Manajer waktu:** estimasi waktu tiap leg = jeda mulai 3 s + translasi + rotasi + tahan 1 s, karena controller asli bergerak dulu baru berputar. Batas waktu tiap percobaan dihitung dari estimasi ini, jadi robot tidak dipotong saat masih berputar. `mission_finished` selalu terkirim sebelum detik ke-288.
7. **Aman:** semua target dibatasi di dalam SafeArea resmi, setiap tunggu punya timeout, dan permintaan `/ib2_user/stop` membatalkan semua aksi.

Hasil uji

Run Gazebo pertama di laptop (program v1.0) memberi 4 checkpoint Level 3 sebelum simulasinya dimatikan watchdog. Dari rekaman lintasannya, model gerak dikalibrasi ulang dan program diperbaiki menjadi v1.1.

Update 31 Okt: di Gazebo asli, v1.1 sudah berhasil 2 dari 3 run dengan 8 checkpoint Level 3 + Goal. Run ketiga gagal di Goal karena navigasi simulator berhenti sendiri, bukan karena program. Program terbaru (v1.2) memperbaiki satu bug perencanaan, dan v1.1 diarsipkan di `04_Solusi/arsip`. Detail ada di `08_Logs/CATATAN_CLAUDE.md`.

Run	Program	Hasil
Gazebo asli, laptop (veryhard)	v1.0	4 checkpoint Level 3 (cp 1, 2, 8, 10) dalam 181 s simulasi, lalu dihentikan watchdog 10 menit jam dinding
Gazebo asli, laptop (veryhard), 3 run berturut-turut	v1.1	Run 1 dan 2: 8 checkpoint Level 3 + Goal (276 s, 252 s). Run 3: 8 checkpoint Level 3, Goal gagal karena node navigasi simulator membeku di t≈223 s
Tiruan, putar ulang ketiga cheat sheet run di atas	v1.2	9, 8 dan 9 checkpoint Level 3 + Goal (240–278 s); bug faktor kecepatan setelah koreksi sudah diperbaiki
Tiruan, cheat sheet yang sama	v1.1	9 checkpoint Level 3 (8 + Goal) dalam 252–270 s
Tiruan veryhard, 12 seed, robot 10% lebih lambat	v1.1	rata-rata 9.3 checkpoint per run (8–10), semuanya Level 3, Goal 12/12, 224–282 s
Tiruan veryhard, robot 25% lebih lambat (6 seed)	v1.1	rata-rata 9 checkpoint, semuanya Level 3, Goal 6/6, maks 287 s
Tiruan normal, 8 checkpoint (6 seed)	v1.1	8/8 Level 3 + Goal di setiap run, rata-rata 244 s
Tiruan easy, 6 checkpoint (6 seed)	v1.1	6/6 Level 3 + Goal di setiap run, rata-rata 198 s

Temuan dari run asli:

- **Posisi sangat akurat**: meleset 4–7 mm, padahal batas Level 3 adalah 10 cm. Gangguan di simulator lokal hanya ±1 cm.
- **Controller bergerak dulu, baru berputar**, tidak bersamaan. Translasi 1.5 m makan ±22 s (puncak ±0.13 m/s), lalu rotasi 100–170° makan 7–10 s.
- Karena itu 12 checkpoint tidak muat dalam 5 menit. Program v1.1 memilih sendiri 8–10 checkpoint terbaik yang masih sempat, ditambah Goal.
- Bug v1.0: batas waktu percobaan terlalu pendek sehingga rotasi terpotong, dan koreksi orientasi relatif membuat robot berputar balik. Keduanya sudah diperbaiki di v1.1.

Cara ujinya: program dijalankan tanpa diubah, checkpoint dibuat oleh generator asli, dan nilai dihitung modul juri asli. Hanya ROS dan gerak robot yang tiruan, dan sekarang model geraknya dikalibrasi dari lintasan Gazebo asli. Tiruan ini masih sedikit lebih cepat (±10%) dari Gazebo, jadi tes dijalankan dengan `--speed 0.9`.

Belum diketahui: rumus pasti pengali waktu. Program memaksimalkan jumlah checkpoint lalu waktu. Kalau ternyata sisa waktu jauh lebih berharga, atur `FINISH_MARGIN` atau batasi jumlah checkpoint.

Alur kerja harian

Urutannya dari yang paling cepat ke yang paling mahal: tes cepat (detik), simulator lokal (±5 menit per run), lalu Web Simulator (jatah 120 run/bulan).

1. **Edit** `~/krpc/krpc7th-simulator/users/catkin_ws/src/krpc_user/scripts/user_template.py` di VS Code (mode WSL). Kalau mengedit salinan di `E:\KIBO Challenge\04_Solusi`, pasang ulang dengan `bash "/mnt/e/KIBO Challenge/04_Solusi/pasang_solusi.sh"`.
2. **Tes cepat tanpa Docker** (program asli + modul juri asli + model gerak tiruan):

   ```
   bash "/mnt/e/KIBO Challenge/06_Tes_Cepat/tes_cepat.sh"             # 1
   run, log lengkap
   bash "/mnt/e/KIBO Challenge/06_Tes_Cepat/tes_cepat.sh" --batch 10   # 10
   run, ringkasan
   ```

   Opsi: `--level easy|normal|hard|veryhard`, `--no-dist` (tanpa gangguan), `--bias 0.2` (gangguan lebih kuat), `--speed 0.6` (robot lebih lambat).
3. **Simulator lokal:** `cd ~/krpc/krpc7th-simulator && bash ./simulation.sh`, lalu pantau di http://localhost. Cek `share/logs/latest/` : `judge_module_logs` untuk skor, `user_programs_logs/user_program_output.log` untuk log program. Di log, cari baris

`SUMMARY` (level dan waktu tiap checkpoint) dan `bias estimate` (seberapa kuat gangguannya). Level kesulitan diatur lewat variabel: `LEVELS=veryhard bash ./simulation.sh` (pilihan: easy/normal/hard/veryhard).

4. **Bikin file submit:** `bash "/mnt/e/KIBO Challenge/04_Solusi/bikin_submission.sh" tim_v1`. File `.tar.gz` muncul di `E:\KIBO Challenge\07_Submission`.
5. **Web Simulator:** login di https://jaxa.krpc.jp/user-auth, buka tab SIMULATION, upload file, pilih Simulator Level, lalu START SIMULATION. Setelah status Finished, buka VIEW RESULT dan DOWNLOAD LOG FILES. Web Simulator berjalan di ARM64 (seperti robot asli), jadi waktunya bisa sedikit berbeda dari laptop.

Laptop lambat? Perpanjang watchdog lokal. Simulator lokal mematikan dirinya setelah 10 menit *jam dinding*. Di laptop i3-N305 Gazebo jalan ±0.3× waktu nyata, jadi 10 menit nyata hanya ±180 detik simulasi dan misi terpotong. Jalankan dengan `execution_time=30m LEVELS=veryhard bash ./simulation.sh`. Skor tidak terpengaruh karena juri dan program sama-sama memakai jam simulasi. Web Simulator berjalan di server JAXA, jadi tidak kena masalah ini.

Simpan setiap versi yang bagus dengan git (`git init` di folder `users`) supaya mudah kembali kalau ada perubahan yang memperburuk skor.

Parameter yang bisa di-tuning

Semua ada di bagian atas `user_template.py`. Ubah satu per satu, lalu bandingkan hasil `--batch` sebelum dan sesudahnya.

Parameter	Nilai	Pengaruh
`HOLD_SEC`	1.0 s	Lama Level 3 harus bertahan sebelum lapor. Rulebook bilang 1 detik, jadi jangan diturunkan tanpa bukti dari Web Simulator.
`MAX_CORRECTIONS`	2	Jumlah percobaan koreksi per checkpoint. Lebih banyak berarti lebih akurat tapi lebih lama.
`BIAS_FF_GAIN`	0.8	Seberapa besar offset gangguan yang dipelajari langsung dikompensasi di percobaan pertama.

Parameter	Nilai	Pengaruh
`FINISH_MARGIN`	12 s	Cadangan waktu sebelum batas 300 detik untuk `mission_finished`.
`PLAN_SAFETY`	1.25	Faktor pengaman estimasi waktu (otomatis disesuaikan dengan kecepatan nyata).
`SETTLE_WINDOW` / `SETTLE_POS`	1.5 s / 0.012 m	Kapan robot dianggap "sudah diam" sehingga perlu dikoreksi.
`V_MAX`, `A_MAX`, `W_MAX`	a 0.0127 m/s², ω 0.436 rad/s, α 0.145 rad/s²	Model gerak perencanaan, dikalibrasi dari run Gazebo asli (awalnya dari `ctl.yaml` Int-Ball2). Translasi dan rotasi dihitung berurutan. Ubah kalau leg di log asli jauh lebih lama/cepat dari estimasi.

Ide pengembangan, diurutkan dari dampak terbesar:

1. **Kalibrasi dengan data asli:** setelah run lokal pertama, bandingkan waktu tiap leg di log dengan estimasi, lalu sesuaikan `V_MAX` / `A_MAX` / `STOP_OVERHEAD`.
2. **Waypoint tanpa berhenti:** controller selalu menahan 3 detik di tiap target. Memakai `/ctl/wrench` bisa menghemat waktu ini, tapi lebih sulit dan berisiko.
3. **Rute sadar orientasi:** biaya putar sudah dihitung, tapi bisa diperhalus dengan model rotasi yang dikalibrasi.
4. **Kamera + ArUco** (OpenCV 3.3.1, kamus 4x4, marker 2 cm) sebagai cadangan kalau navigasi meleset. Juri tidak memakai kamera, jadi ini prioritas rendah.

Isi folder E:\KIBO Challenge

Folder / file	Isi
`00_BACA_DULU.md`	Ringkasan singkat + link ke guide ini
`download_semua.bat`	Unduh ulang semua file resmi dari jaxa.krpc.jp
`01_Dokumen`	Guidebook, Rulebook, Beginner Rulebook, Programming Manual, materi Guidance, poster

Folder / file	Isi
`02_Simulator`	`krpc7th-simulator.tar.gz` (simulator lokal, berisi juga `docs/ProgrammingManual.md`)
`03_Sample_Code`	Kode contoh resmi + versi ekstrak di `preview` untuk dibaca
`04_Solusi`	`user_template.py` (program kita), `pasang_solusi.sh`, `bikin_submission.sh`
`05_Setup`	`0_install_wsl_ubuntu.bat` (Windows), `1_setup_ubuntu.sh` (Ubuntu)
`06_Tes_Cepat`	`tes_cepat.sh` + harness tiruan (`run_mock.py`, `mockros.py`)
`07_Submission`	Hasil `bikin_submission.sh`, siap diupload

Di root folder ada salinan PDF dan `.tar.gz` yang sama persis dengan isi `01_Dokumen` / `02_Simulator` (sisa unduhan browser). Aman dihapus.

Sumber

- Kibo-RPC — situs resmi (jadwal, POC, deadline 4 Nov 2026 14:59)
- Halaman Download dan FAQ
- 7th Kibo-RPC Rulebook v1.1 (15 Okt 2026) dan Programming Manual v1.0 (5 Okt 2026), keduanya di `01_Dokumen`
- Kode juri `rootfs/krpc/judge_module/judge.py` dan generator checkpoint `generate_objects.py` di dalam paket simulator
- jaxa/int-ball2_simulator branch melodic, `ctl.yaml` (kecepatan, toleransi, waktu tahan controller)